CSIS 402 : COMPUTER ORGANIZATION & SYSTEM PROGRAMMING

LECTURE 4

Dr. Sarah Azzam

Slides Based on Lecture Slides by Prof. Dr. Amr Talaat

FUNDAMENTAL MARIE INSTRUCTIONS

The fundamental MARIE instructions are:

Instruction Number		Instruction → Meaning
Bin	**Hex**	
0001	**1**	**Load X** → Load the contents of address X into AC.
0010	**2**	**Store X** → Store the contents of AC at address X.
0011	**3**	**Add X** → Add the contents of address X to AC and store the result in AC.
0100	**4**	**Subt X** → Subtract the contents of address X from AC and store the result in AC.
0101	**5**	**Input** → Input a value from the keyboard into AC.
0110	**6**	**Output** → Output the value in AC to the display.
0111	**7**	**Halt** → Terminate the program.
1000	**8**	**Skipcond** → Skip the next instruction on condition. (While, If)
1001	**9**	**Jump X** → Load the value of X into PC

EXAMPLE 1: ASSEMBLY PROGRAM

Write an assembly program in Marie that:

- Takes two inputs
- Store the inputs into two variables X and Y
- Add the two variables and store the result to Z
- Display the result to the Console

EXAMPLE 1: ASSEMBLY PROGRAM

Assume X=7,Y=5

Assembly code:

Autosaved file

```
1  Input
2  Store X
3  Input
4  Store Y
5  Load X
6  Add Y
7  Store Z
8  Output
9  Halt
10 X, Dec 0
11 Y, Dec 0
12 Z, Dec 0
13
14
```

Input Value

Please input a value.

Value:

7

Type:

Hexadecimal

Cancel and pause | Accept

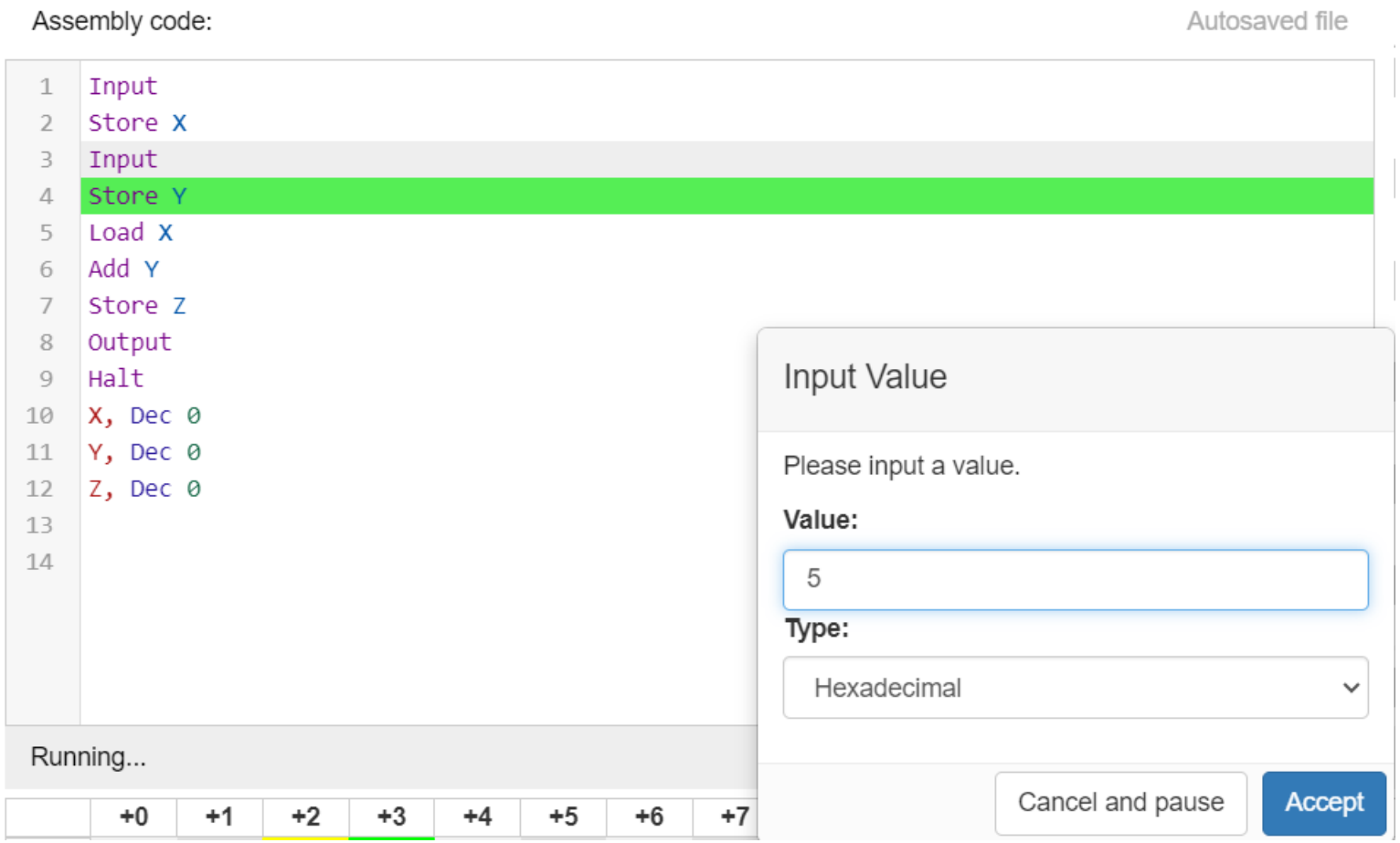
Running...

	+0	+1	+2	+3	+4	+5	+6	+7
000	5009	3009	5008	2007	1009	2007	200B	6000

Ref: https://marie.js.org/

EXAMPLE 1: ASSEMBLY PROGRAM

Assume X=7,Y=5



Ref: https://marie.js.org/

EXAMPLE 1: ASSEMBLY PROGRAM

Assembly code:

Assume X=7,Y=5

```
1  Input
2  Store X
3  Input
4  Store Y
5  Load X
6  Add Y
7  Store Z
8  Output
9  Halt
10 X, Dec 0
11 Y, Dec 0
12 Z, Dec 0
13
14
```

Machine halted normally.

The value saved in the AC

Instruction Register Content

Autosaved file

AC
IR
MAR
MBR
PC
IN
OUT

Output log | RTL log | Watch list | Input list

OUTPUT MODE: HEX

000C

Output displayed on the Console

Instruction Number		Instruction → Meaning
Bin	Hex	
0001	1	**Load X** → Load the contents of address X into AC.
0010	2	**Store X** → Store the contents of AC at address X.
0011	3	**Add X** → Add the contents of address X to AC and store the result in AC.
0100	4	**Subt X** → Subtract the contents of address X from AC and store the result in AC.
0101	5	**Input** → Input a value from the keyboard into AC.
0110	6	**Output** → Output the value in AC to the display.
0111	7	**Halt** → Terminate the program.
1000	8	**Skipcond** → Skip the next instruction on condition.
1001	9	**Jump X** → Load the value of X into PC

Ref: https://marie.js.org/

EXAMPLE 1: ASSEMBLY PROGRAM

Assembly code:

Assume X=7,Y=5

```
1  Input
2  Store X
3  Input
4  Store Y
5  Load X
6  Add Y
7  Store Z
8  Output
9  Halt
10 X, Dec 0
11 Y, Dec 0
12 Z, Dec 0
13
14
```

Autosaved file

The value saved in the AC

Instruction Register Content

AC	000C
IR	7000
MAR	008
MBR	7000
PC	009
IN	0005
OUT	000C

Output log | RTL log | Watch list | Input list

OUTPUT MODE: HEX

000C

Output displayed on the Console

Machine halted normally.

Instruction Number		Instruction → Meaning
Bin	Hex	
0001	1	**Load X** → Load the contents of address X into AC.
0010	2	**Store X** → Store the contents of AC at address X.
0011	3	**Add X** → Add the contents of address X to AC and store the result in AC.
0100	4	**Subt X** → Subtract the contents of address X from AC and store the result in AC.
0101	5	**Input** → Input a value from the keyboard into AC.
0110	6	**Output** → Output the value in AC to the display.
0111	7	**Halt** → Terminate the program.
1000	8	**Skipcond** → Skip the next instruction on condition.
1001	9	**Jump X** → Load the value of X into PC

Ref: https://marie.js.org/

EXAMPLE 2: ASSEMBLY PROGRAM

Write a MARIE assembly language program to compute the first 100 elements of the following sequence:
$F_0 = 0, F_1 = 1, F_2 = 3, F_i = F_{i-1} + F_{i-2} + F_{i-3}$, for i > 2.

You should display the 100 elements of this sequence.

Illustrating example:

$F_3 = F_0 + F_1 + F_2 = 0 + 1 + 3 = 4$

$F_4 = F_1 + F_2 + F_3 = 1 + 3 + 4 = 8$

Try it on MARIE Simulator → Ref: https://marie.js.org/

EXAMPLE 2: ASSEMBLY PROGRAM

/ MARIE Assembly Program to generate and output the first 100 terms of the Tribonacci sequence

Assembly Code

```
/
F0,      DEC     0          / Initial value for F0
F1,      DEC     1          / Initial value for F1
F2,      DEC     3          / Initial value for F2
Next,    DEC     0          / Temporary Variable for storing
Counter,  DEC   97          / Remaining terms to be calculated and printed
One,     DEC     1
```

EXAMPLE 2: ASSEMBLY PROGRAM

```
/ MARIE Assembly Program to generate and output the first 100 terms of the Tribonacci
sequence
```

```
/ Output the first three terms
        Load    F0
        Output
        Load    F1
        Output
        Load    F2
        Output
```

Section I

EXAMPLE 2: ASSEMBLY PROGRAM

```
/ MARIE Assembly Program to generate and output the first 100 terms of the Tribonacci sequence

/Calculate and output the remaining terms from F3 till F97

Loop,   Load    Counter
        Skipcond 800        / If Counter > 0, continue loop
        Halt                / If Counter <= 0, end the program

        Load    F0
        Add     F1
        Add     F2
        Store   Next        / Store computed term in Next
        Output

/ Update the variables needed for computation for the next iteration
        Load    F1          / F0 ←F1
        Store   F0
        Load    F2          / F1 ← F2
        Store   F1
        Load    Next        / F2 ← Next
        Store   F2
```

Section II

EXAMPLE 2: ASSEMBLY PROGRAM

```
/ MARIE Assembly Program to generate and output the first 100 terms of the Tribonacci sequence

/Calculate and output the remaining terms from F3 till F97

Loop,   Load    Counter
        Skipcond 800        / If Counter > 0, continue loop
        Halt                / If Counter <= 0, end the program

:
:
:
:
:
:
/ Decrement Counter
        Load    Counter
        Subt    One
        Store   Count

        Jump    Loop
```

Loop Body

Section III

EXAMPLE 2: ASSEMBLY PROGRAM

```
/ MARIE Assembly Program to generate and output the first 100
terms of the Tribonacci sequence


/ Output the first three terms
        Load    F0
        Output
        Load    F1
        Output
        Load    F2
        Output

/Calculate and output the remaining terms from F3 till F97

Loop,   Load    Counter
        Skipcond 800        / If Counter > 0, continue loop
        Halt                / If Counter <= 0, end the program

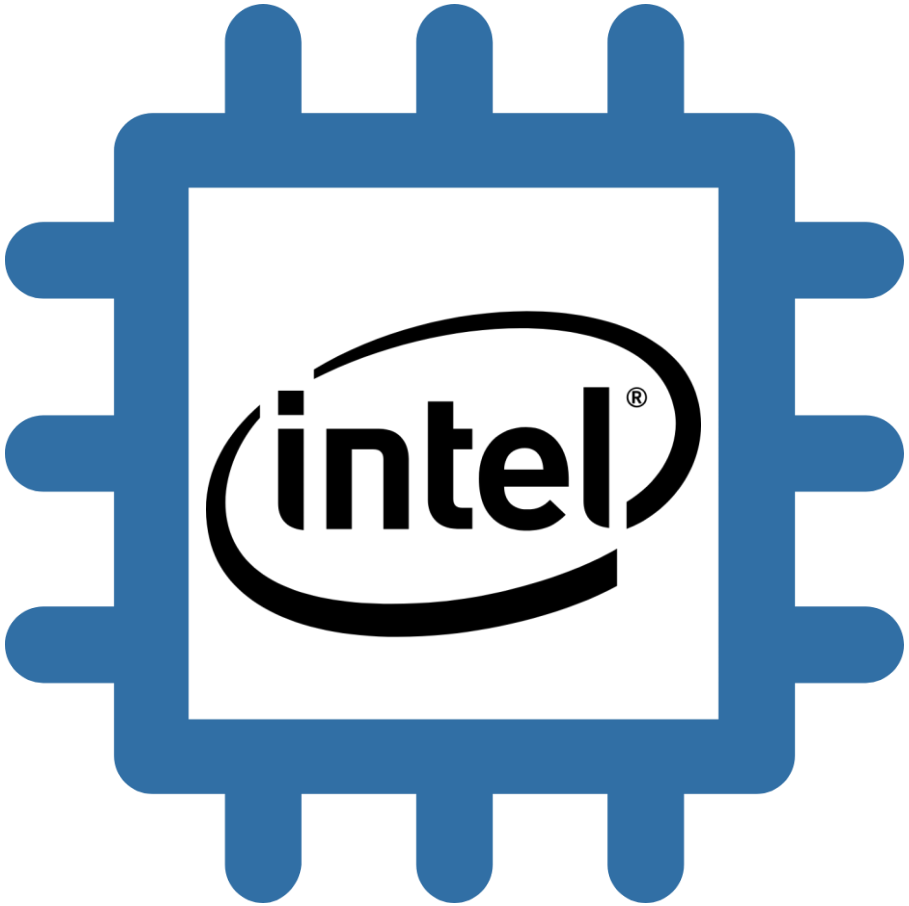
        Load    F0
        Add     F1
        Add     F2
        Store   Next        / Store computed term in Next
        Output
```



```
/ Update the variables needed for computation for the next iteration
        Load    F1          / F0 ←F1
        Store   F0
        Load    F2          / F1 ← F2
        Store   F1
        Load    Next        / F2 ← Next
        Store   F2

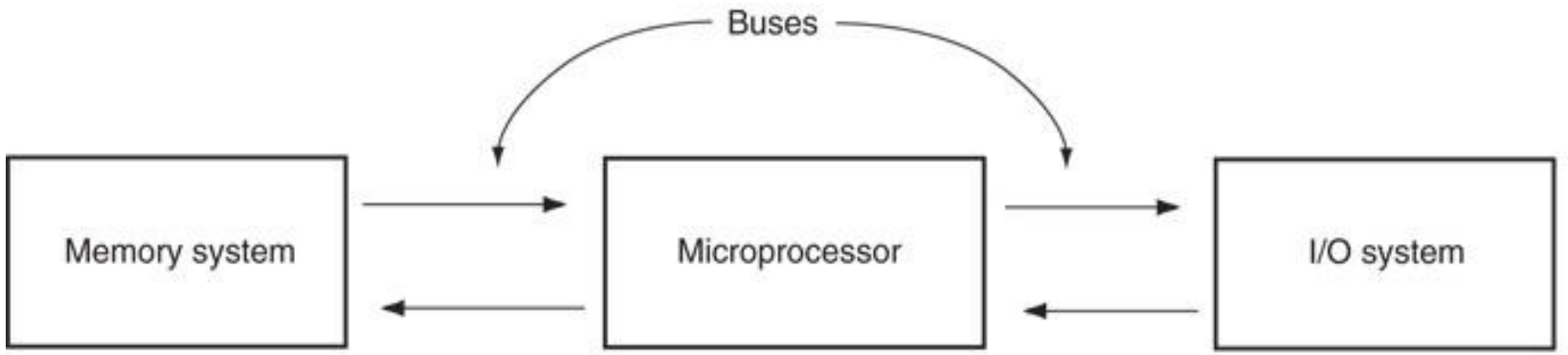
/ Decrement Counter
        Load    Counter
        Subt    One
        Store   Count

        Jump    Loop
/
F0,     DEC     0           / Initial value for F0
F1,     DEC     1           / Initial value for F1
F2,     DEC     3           / Initial value for F2
Next,   DEC     0           / Temporary Variable for storing
Counter,  DEC  97           / Remaining terms to be calculated and printed
One,    DEC     1
```



INTEL 80x86 MICROPROCESSOR

THE BLOCK DIAGRAM OF A MICROPROCESSOR-BASED COMPUTER SYSTEM



Dynamic RAM (DRAM)
Static RAM (SRAM)
Cache
Read-only (ROM)
Flash memory
EEPROM
SDRAM
RAMBUS
DDR DRAM

8086
8088
80186
80188
80286
80386
80486
Pentium
Pentium Pro
Pentium II
Pentium III
Pentium 4
Core2

Printer
Serial communications
Floppy disk drive
Hard disk drive
Mouse
CD-ROM drive
Plotter
Keyboard
Monitor
Tape backup
Scanner
DVD

INTEL MICROPROCESSOR EVOLUTION

Product	8080	8085	8086	8088	80286	80386	80486
Year Introduced	1974	1976	1978	1979	1982	1985	1989
Clock Rate (MHz)	2 - 3	3 - 8	5 - 10	5 - 8	6 - 16	16 - 33	25 - 50
No. Transistors	4500	6500	29000	29000	130000	275000	1.2 M
Physical Memory	64K	64K	1M	1M	16M	4G	4G
Internal data bus	8	8	16	16	16	32	32
External data bus	8	8	16	8	16	32	32
Address bus	16	16	20	20	24	32	32
Data type (bits)	8	8	8, 16	8, 16	8, 16	8, 16, 32	8, 16, 32

IMPROVEMENTS OF INTEL 8086 OVER 8080/8085

List Three Improvements of Intel 8086 over 8080/8085.

1. The memory capacity of 8086 is 1MB.
2. 8086 is a 16-bit processor.
3. 8086 is a pipelined processor.

Product	8080	8085	8086
Year Introduced	1974	1976	1978
Clock Rate (MHz)	2 - 3	3 - 8	5 - 10
No. Transistors	4500	6500	29000
Physical Memory	64K	64K	1M
Internal data bus	8	8	16
External data bus	8	8	16
Address bus	16	16	20
Data type (bits)	8	8	8, 16

THE MODERN MICROPROCESSOR

In 1978 Intel released the **8086**; a year or so later, it released the **8088**.

- Both devices are 16-bit microprocessors.
  - executed instructions in as little as **400 ns (2.5 millions of instructions per second)**
  - major improvement over **execution speed** of 8085
- **8086** & **8088** addressed **1M byte of memory**.
  - **16 times** more memory than the 8085
  - **1M-byte memory** contains 1024K byte-sized memory locations or 1,048,576 bytes

THE MODERN MICROPROCESSOR

- **Increased memory size** and **additional instructions** in **8086**/**8088** led to many sophisticated applications.
- **Improvements** to the instruction set included **multiply and divide instructions**.
  - missing on earlier microprocessors
- Number of instructions increased.
  - from 45 on the 4004, to 246 on the 8085
  - over 20,000 variations on the **8086** & **8088**

THE MODERN MICROPROCESSOR

- These microprocessors are called **CISC (complex instruction set computers)**.
  - additional instructions eased task of developing efficient and sophisticated applications

INTEL MICROPROCESSOR EVOLUTION

Product	8080	8085	8086	8088	80286	80386	80486
Year Introduced	1974	1976	1978	1979	1982	1985	1989
Clock Rate(MHz)	2 - 3	3 - 8	5 - 10	5 - 8	6 - 16	16 - 33	25 - 50
No. Transistors	4500	6500	29000	29000	130000	275000	1.2 M
Physical Memory	64K	64K	1M	1M	16M	4G	4G
Internal data bus	8	8	16	16	16	32	32
External data bus	8	8	16	8	16	32	32
Address bus	16	16	20	20	24	32	32
					, 16	8, 16, 32	8, 16, 32

8086 microprocessor with a 16-bit internal and external data bus →
- All registers are 16 bits wide
- There is a 16 bits bus to transfer data in and out of the CPU

INTEL MICROPROCESSOR EVOLUTION

Product	8080	8085	8086	8088	8		80486
Year Introduced	1974	1976	1978	1979			1989
Clock Rate(MHz)	2 - 3	3 - 8	5 - 10	5 - 8		33	25 - 50
No. Transistors	4500	6500	29000	29000	130000	275000	1.2 M
Physical Memory	64K	64K	1M	1M	16M	4G	4G
Internal data bus	8	8	16	16	16	32	32
External data bus	8	8	16	8	16	32	32
Address bus	16	16	20	20	24	32	32
Data type (bits)	8	8	8, 16	8, 16	8, 16	8, 16, 32	8, 16, 32

Why do you think the external data bus were 8 bits again in the newer 8088?

INTEL 8088 VS 8086 MICROPROCESSOR

Despite the great advancement of 8086 over the earlier versions but there were some limitations using 16-bit external data bus.

These limitations were due to:

- **peripherals** were designed around an **8-bit processor** at that time.

- a printed circuit board with a 16-bit data bus was of a **higher cost**.

8088 was similar to the 8086 except for the external data bus.

Popularity of Intel ensured in 1981 when IBM chose the 8088 in its personal computer.

INTEL MICROPROCESSOR EVOLUTION

Virtual Memory

Product	8080	8085	8086	8088	80286	80386	80486
Year Introduced	1974	1976	1978	1979	1982	1985	1989
Clock Rate(MHz)	2 - 3	3 - 8	5 - 10	5 - 8	6 - 16	16 - 33	25 - 50
No. Transistors	4500	6500	29000	29000	130000	275000	1.2 M
Physical Memory	64K	64K	1M	1M	16M	4G	4G
Internal data bus	8	8	16	16	16	32	32
External data bus	8	8	16	8	16	32	32
Address bus	16	16	20	20	24	32	32
Data type (bits)	8	8	8, 16	8, 16	8, 16	8, 16, 32	8, 16, 32

These microprocessors were developed due to the need of more powerful microprocessors.

VIRTUAL MEMORY

Virtual Memory

- A **method of fooling** the microprocessor to infer that it can **access nearly unlimited amount of memory**.
- This is done by **swapping data** between **disk storage** and **RAM**.

INTEL MICROPROCESSOR EVOLUTION

Cache Memory

Product	8080	8085	8086	8088	80286	80386	80486
Year Introduced	1974	1976	1978	1979	1982	1985	1989
Clock Rate(MHz)	2 - 3	3 - 8	5 - 10	5 - 8	6 - 16	16 - 33	25 - 50
No. Transistors	4500	6500	29000	29000	130000	275000	1.2 M
Physical Memory	64K	64K	1M	1M	16M	4G	4G
Internal data bus	8	8	16	16	16	32	32
External data bus	8	8	16	8	16	32	32
Address bus	16	16	20	20	24	32	32
Data type (bits)	8	8	8, 16	8, 16	8, 16	8, 16, 32	8, 16, 32

CACHE MEMORY

Cache Memory

- A **static RAM** with **a very fast access time**.
- It holds the **data** and **programs** that the CPU uses frequently.
- It **speeds up the system performance** because if the required instructions or data are in the system's cache memory, the CPU does not need to access the primary memory or RAM.

PIPELINING

- Instructions are stored in memory → So it is fetched, decoded and then executed.
- Earlier than 8086, the CPU can either fetch or execute in a specific instance



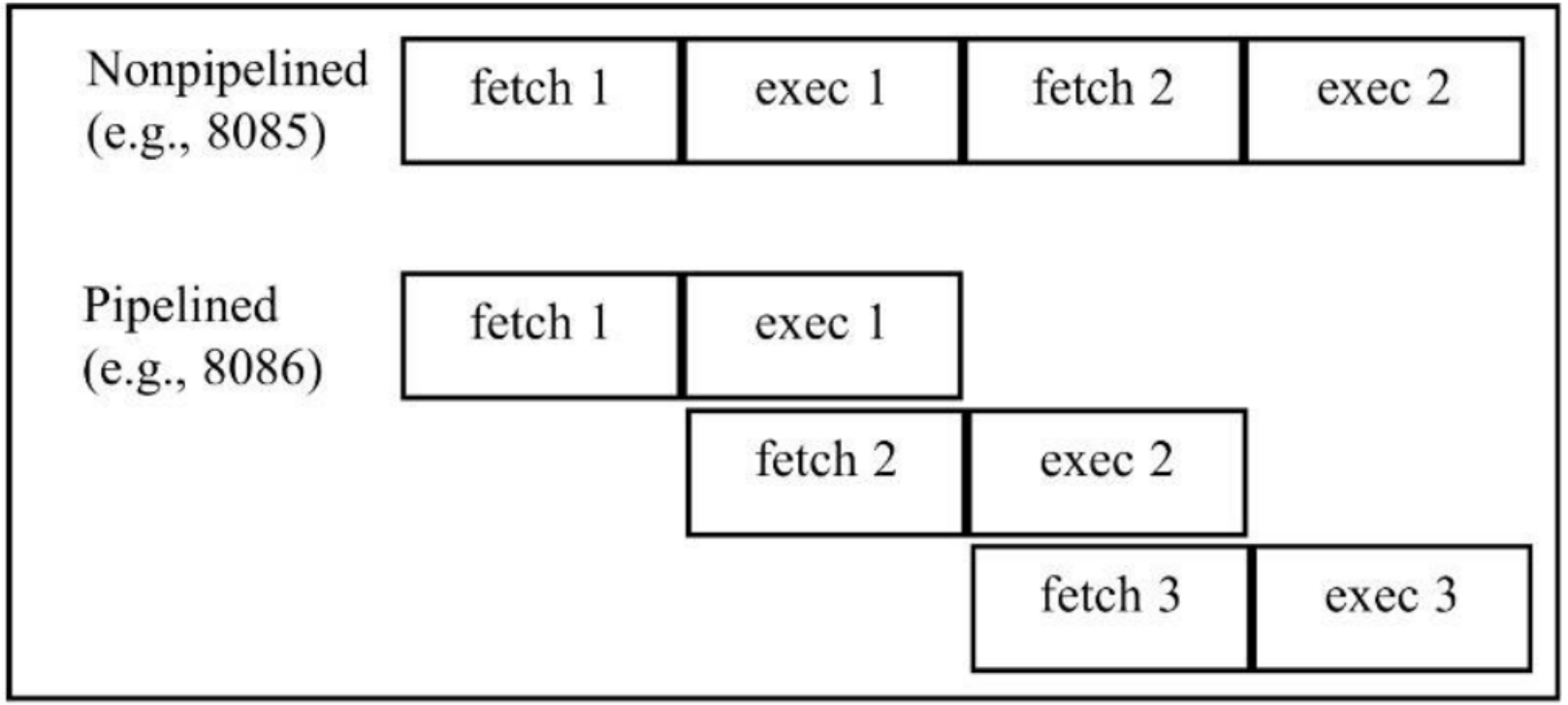
Pipelining

- is to allow the CPU to **fetch the next instruction** while **executing the current instruction**.
- Pipelining leads to **performance improvement** → Since **fetching** and **executing** is done at the **same time**.

How is the pipelining implemented in

Intel 8086/8088 ?

PIPELINED VS. NON-PIPELINED EXECUTION

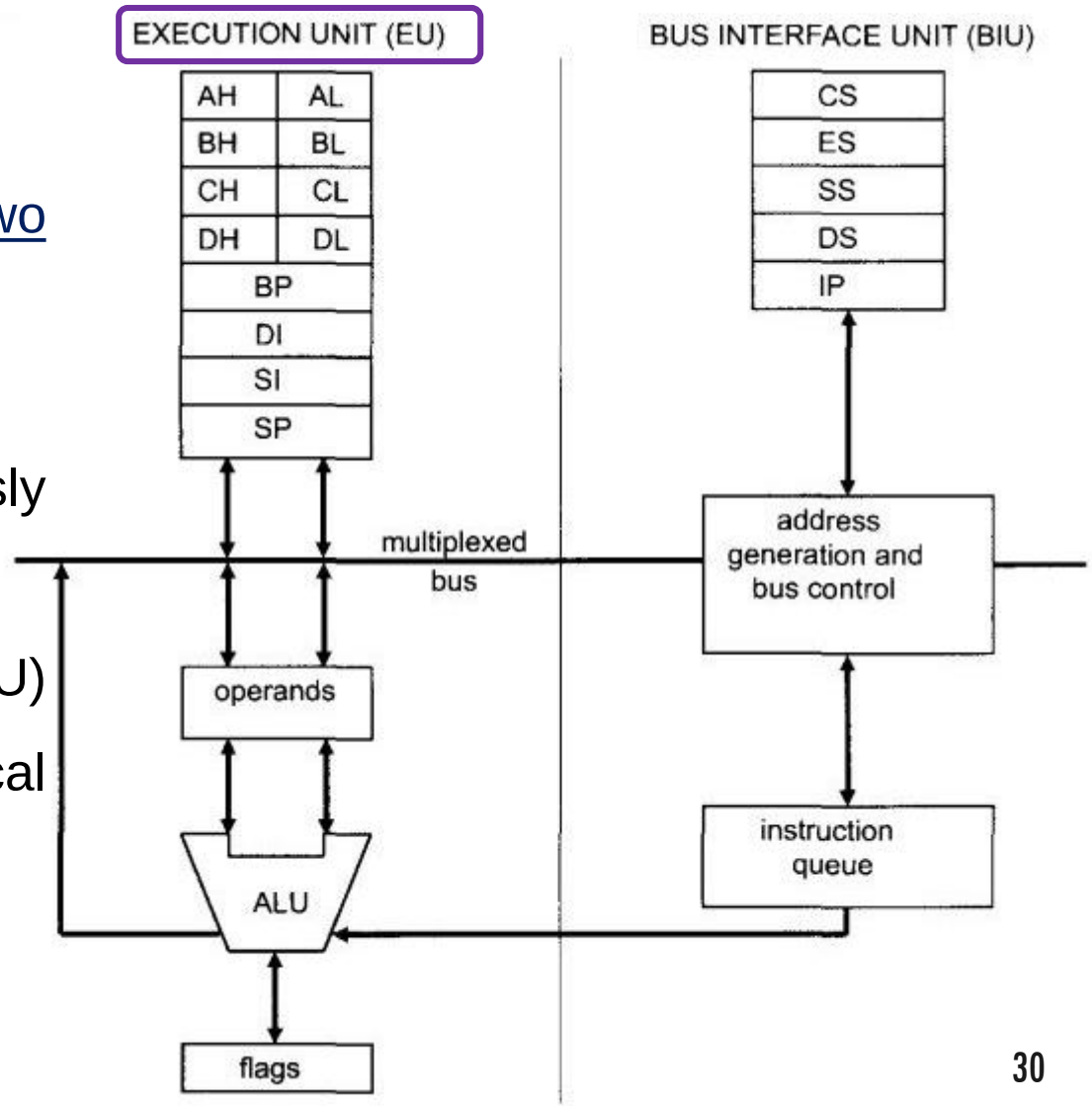


INTERNAL STRUCTURE OF THE 8086 MICROPROCESSOR

The internal structure is divided into two sections:

- Execution Unit (EU):

- Executes instructions that are previously fetched.
- Contains the arithmetic logic unit (ALU) which performs arithmetic and logical operations.

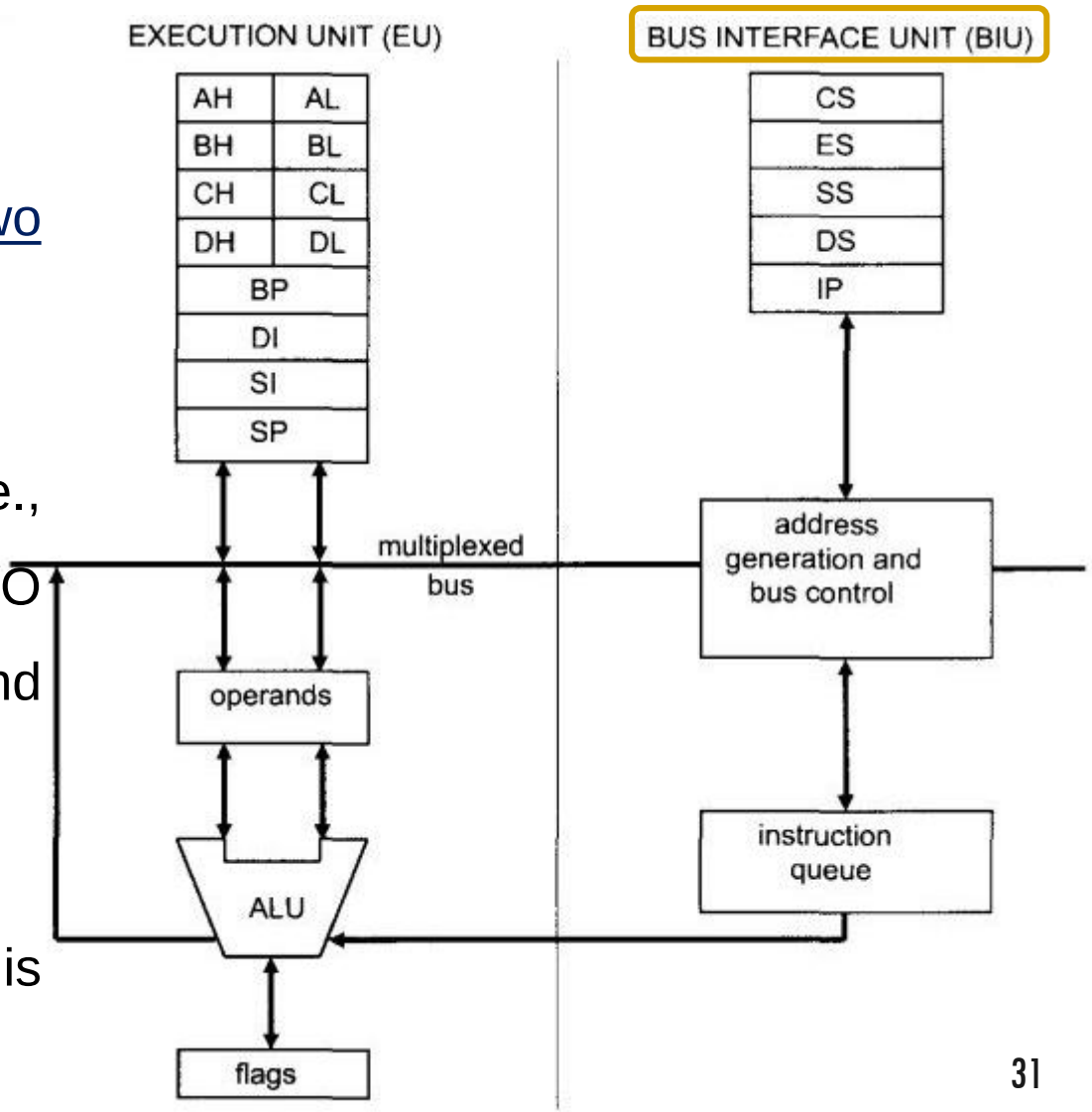


INTERNAL STRUCTURE OF THE 8086 MICROPROCESSOR

The internal structure is divided into two sections:

- Bus Interface Unit (BIU):

- Accesses memory and peripherals i.e., perform memory read-write operations, I/O read transfers data between memory and I/O devices.
- Has a buffer/queue (6 bytes long)
- Fetches a new instruction when there is room for 2 bytes.



A Jump instruction is executed.

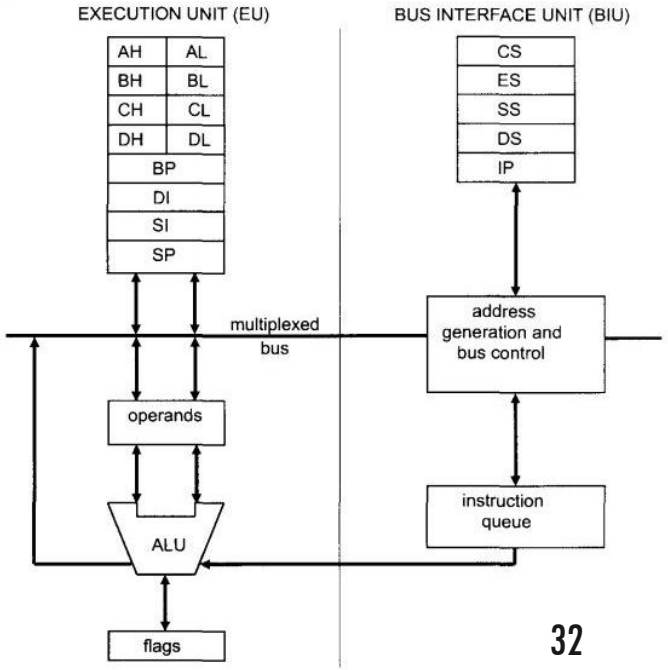
An instruction took long time to execute.

Branch penalty

- Previously fetched information in the queue is discarded.
- BIU fetches information from a new memory location.
- EU waits for the BIU to fetch the new instruction.



The queue reaches its maximum capacity and the buses will be idle.

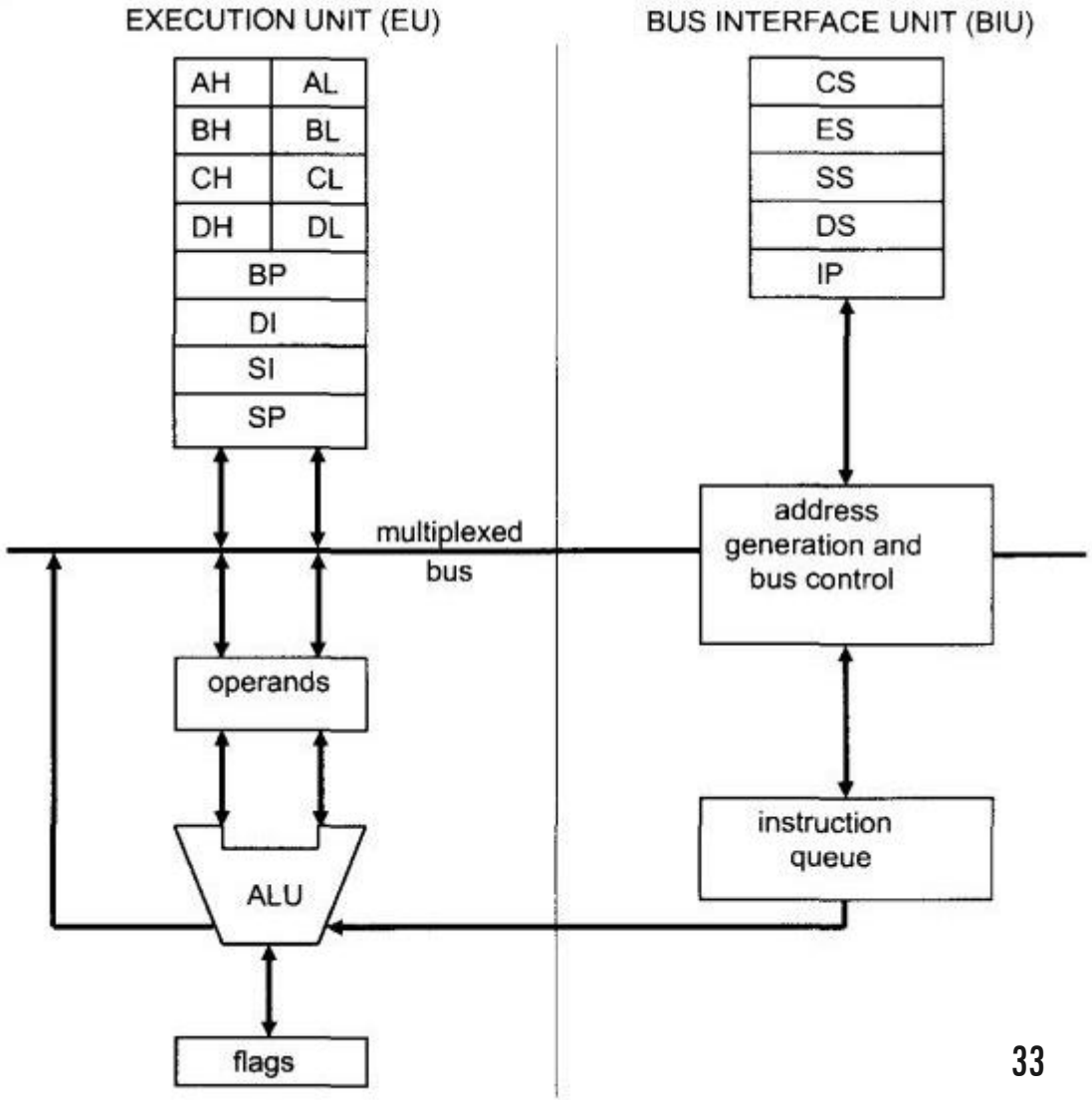


REGISTERS

Registers

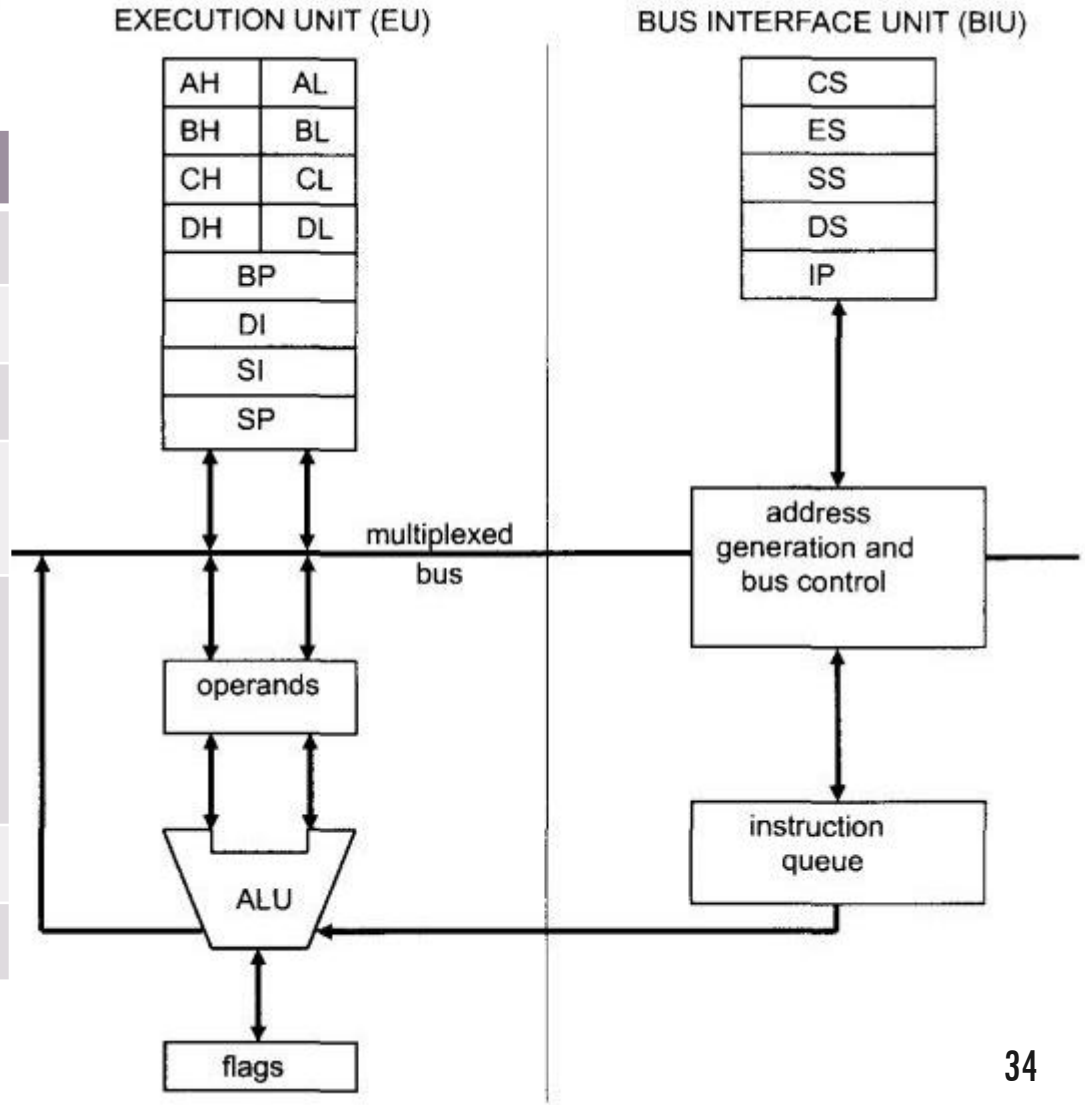
- are used to store information temporarily in the CPU.

Let's discuss Registers in Intel 8086 Microprocessor.



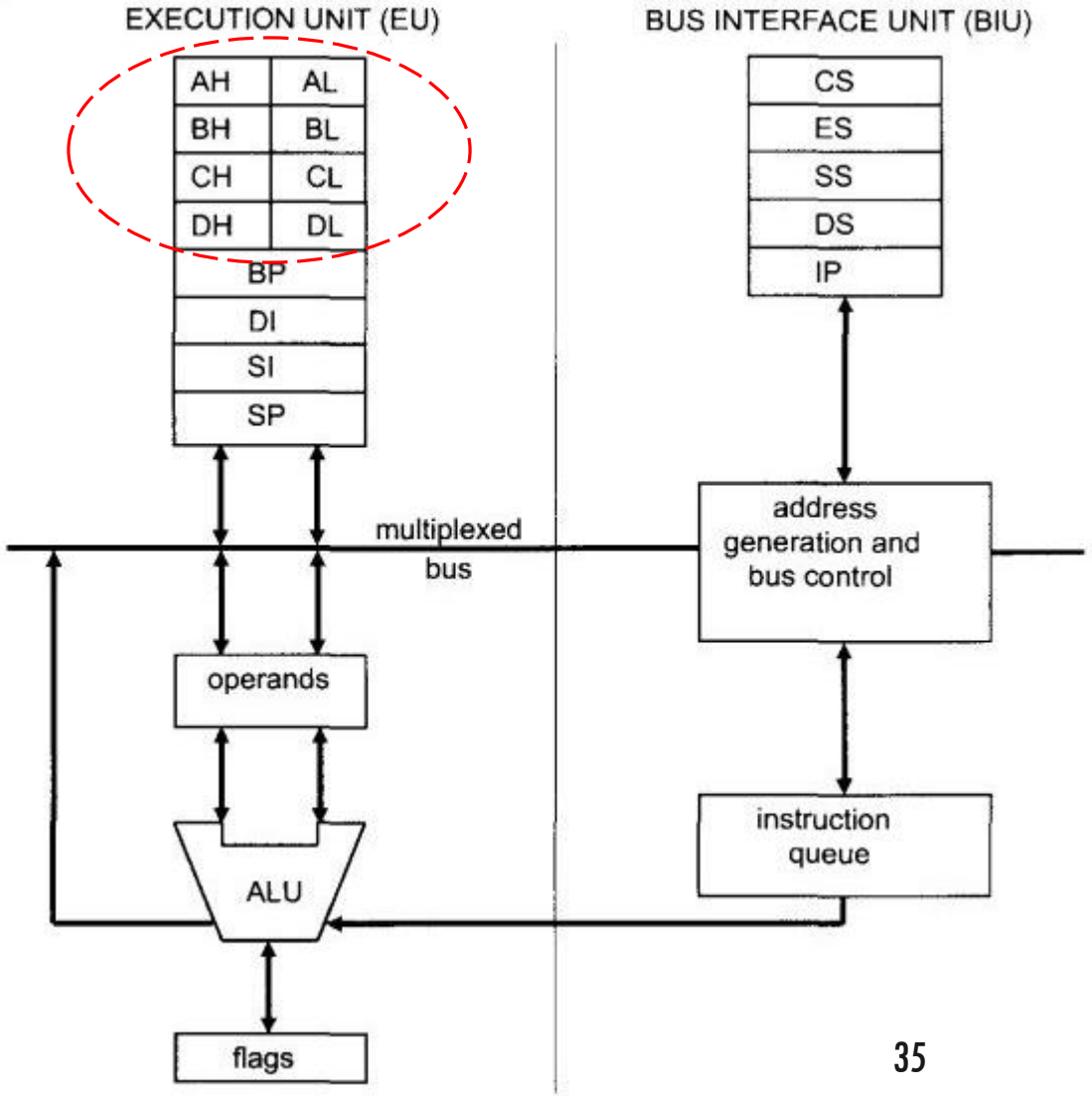
8086 REGISTERS

Category	Bits	Register Names
General	16	AX, BX, CX, DX
	8	AH, AL, BH, BL, CH, CL, DH, DL
Pointer	16	SP (stack pointer), BP (Base Pointer)
Index	16	SI (source index), DI (destination index)
Segment	16	CS (Code Segment), DS (Data Segment), SS (Stack Segment), ES (Extra Segment)
Instruction	16	IP (Instruction Pointer)
Flag	16	FR (flag register)



GENERAL PURPOSE REGISTERS

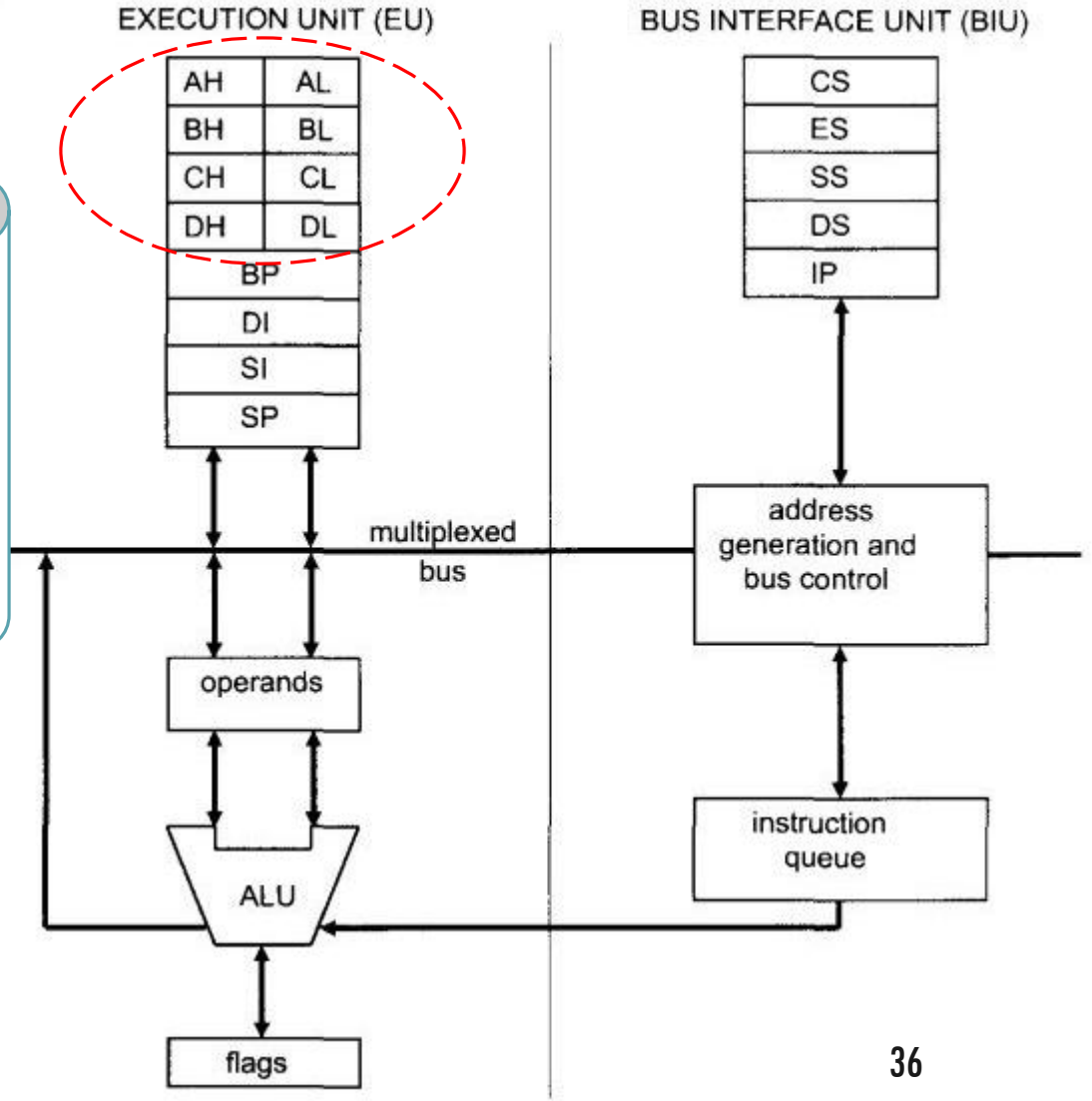
AX (used for Accumulator) 16-bit Register	
AH 8-bit Register	AL 8-bit Register
BX (used as Base Addressing Register) 16-bit Register	
BH 8-bit Register	BL 8-bit Register
CX (used as Counter in loops) 16-bit Register	
CH 8-bit Register	CL 8-bit Register
DX (used to point to data in I/O operations) 16-bit Register	
DH 8-bit Register	DL 8-bit Register



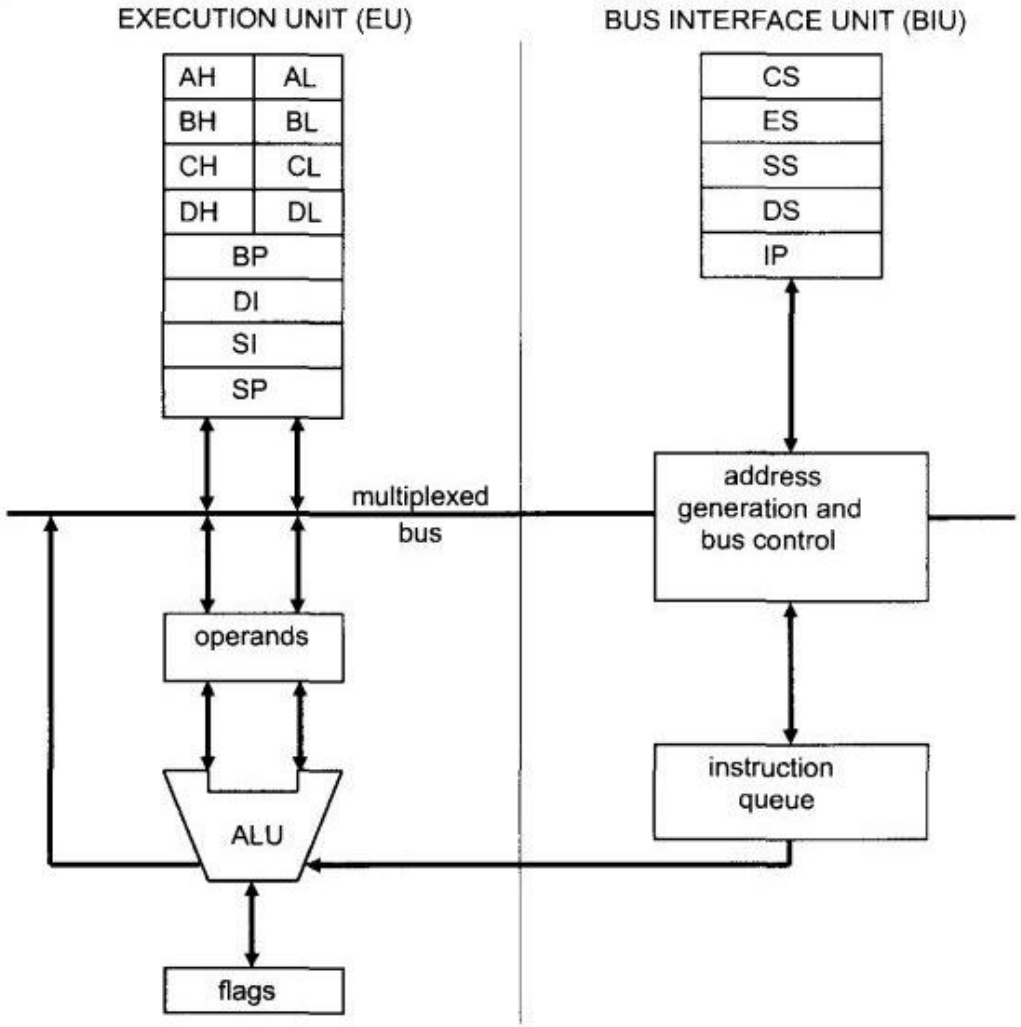
GENERAL PURPOSE REGISTERS

General Purpose Registers can be accessed as:

- 16-bit register (e.g., AX, BX, CX, DX)
- High Byte only (e.g., AH, BH, CH, DH)
- Low Byte only (e.g., AL, BL, CL, DL)

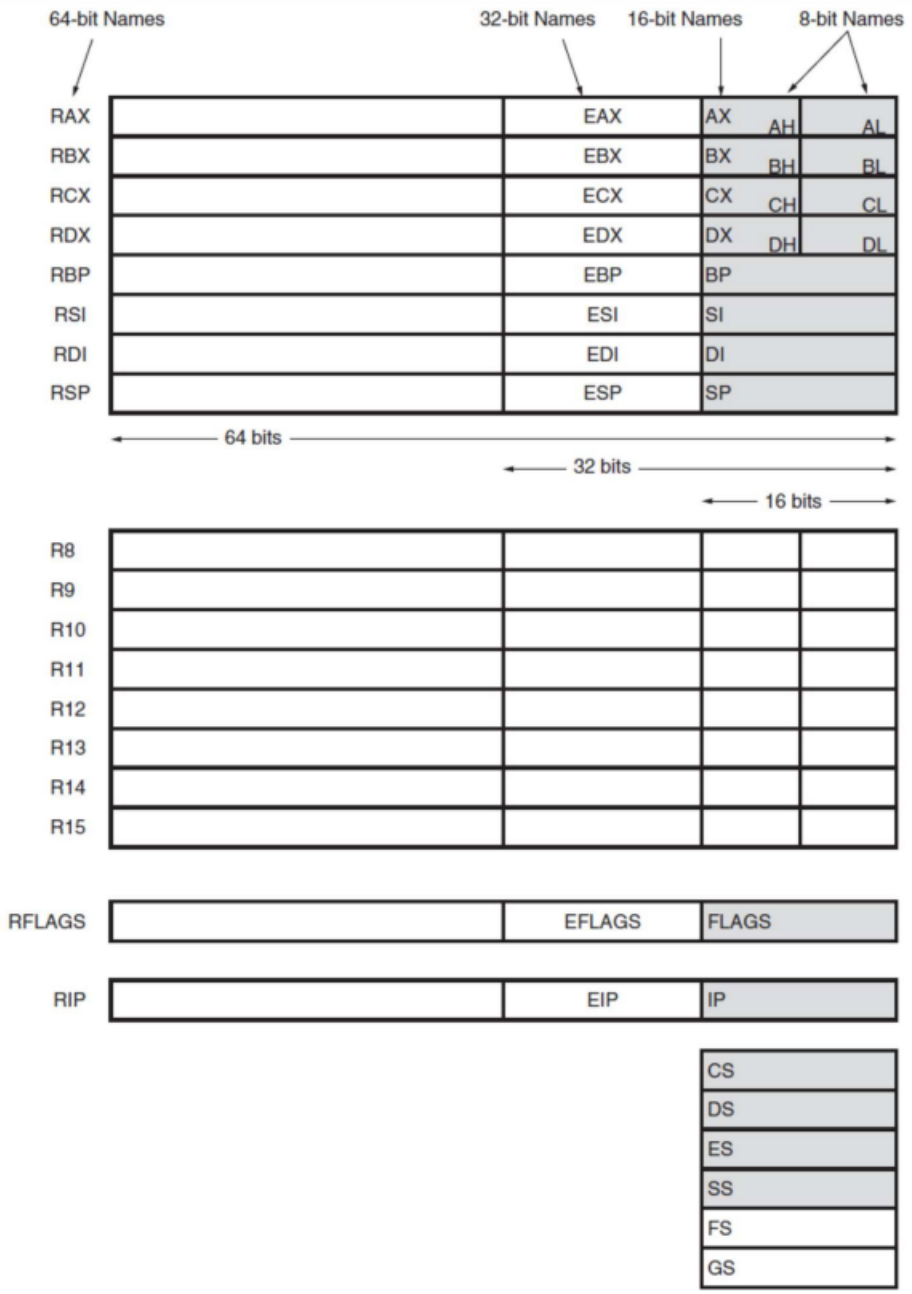


INTERNAL STRUCTURE OF THE 8086 MICROPROCESSOR

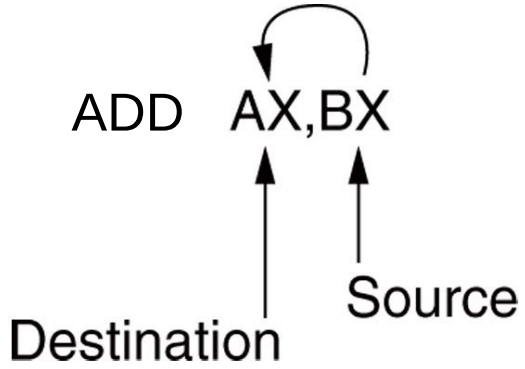


REGISTERS

- Different names are given for 64-bit, 32-bit, 16-bit and 8-bit version of the same register.



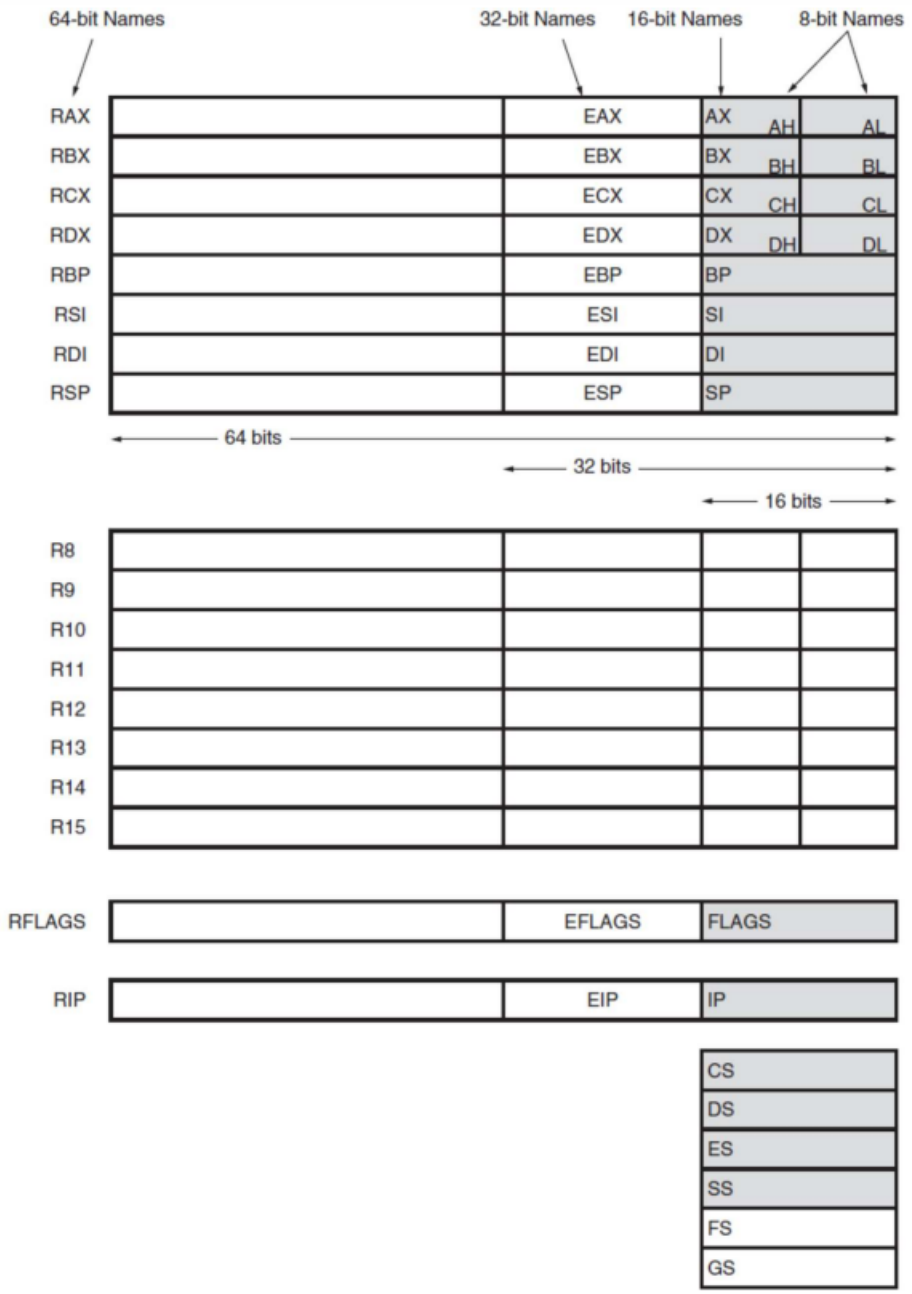
EXAMPLES



1
- Add AL, AH
- The instruction adds the 8-bit contents of AH to AL.
- Only the contents of AL changes based on the previous instruction

2
- Add DX, CX
- The instruction adds the 16-bit contents of CX to DX.
- Only the contents of DX changes based on the previous instruction

3
- Add ECX, EBX
- The instruction adds the 32-bit contents of EBX to ECX.
- Only the contents of ECX changes based on the previous instruction



REFERENCES

[1] https://marie.js.org/
[2] https://ecs-network.serv.pacific.edu/past-courses/2012-spring-ecpe-170/slides/15MARIEsimulator1.pdf (additional problems)